Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VII

```
# To create tree
tree = {}
path = []
maxHeight, maxHeightNode = -1, -1

# Function to store the path from given vertex to the target vertex in a vector path
```

```
def getDiameterPath(vertex, targetVertex, parent, path):

    # If the target node is found, push it into path vector
    if (vertex == targetVertex):
        path.append(vertex)
        return True

    for i in range(len(tree[vertex])):
        # To prevent visiting a node already visited
        if (tree[vertex][i] == parent):
            continue

        # Recursive call to the neighbours of current node inorder to get the path
        if (getDiameterPath(tree[vertex][i], targetVertex, vertex, path)):
            path.append(vertex)
            return True
    return False
```

```
# Function to obtain and return the farthest node from a given vertex
def farthestNode(vertex, parent, height):
    global maxHeight, maxHeightNode

    # If the current height is maximum so far, then save the current node
    if (height > maxHeight):
        maxHeight = height
        maxHeightNode = vertex

    # Iterate over all the neighbours of current node
    if (vertex in tree):
        for i in range(len(tree[vertex])):

            # This is to prevent visiting a already visited node
            if (tree[vertex][i] == parent):
                continue

            # Next call will be at 1 height higher than our current height
            farthestNode(tree[vertex][i], vertex, height + 1)
```

```
# Function to add edges
def addedge(a, b):
    if (a not in tree):
        tree[a] = []

    tree[a].append(b)

    if (b not in tree):
        tree[b] = []

    tree[b].append(a)
```

```
def FindCenter(n):
    # Now we will find the 1st farthest node from 0(any arbitrary node)

    # Perform DFS from 0 and update the maxHeightNode to obtain the farthest node from 0

    # Reset to -1
    maxHeight = -1

    # Reset to -1
    maxHeightNode = -1

    farthestNode(0, -1, 0)
```

```
        # Stores one end of the diameter
        leaf1 = maxHeightNode

        # Similarly the other end of the diameter
        # Reset the maxHeight
        maxHeight = -1
        farthestNode(maxHeightNode, -1, 0)

        # Stores the second end of the diameter
        leaf2 = maxHeightNode

        # Store the diameter into the vector path
        path = []

        # Diameter is equal to the path between the two farthest nodes leaf1 and leaf2
        getDiameterPath(leaf1, leaf2, -1, path)

        pathSize = len(path)

        if (pathSize % 2 == 1):
            print(path[int(pathSize / 2)]*-1)
        else:
            print(path[int(pathSize / 2)], ", ", path[int((pathSize - 1) / 2)], sep = "", end = "")
```

```
N = 4

tree = {}
addedge(1, 0)
addedge(1, 2)
addedge(1, 3)

FindCenter(N)

# This code is contributed by suresh07.
```

```
1
```

Saya akan analisis kode ini dulu secara menyeluruh sebelum menjelaskan logikanya.

Analisis Logika Program

Program ini mencari **center (pusat) dari sebuah pohon (tree)** menggunakan algoritma berbasis diameter pohon.

Alur kerja:

1. Bangun tree sebagai adjacency list
2. Dari node sembarang (0), cari node terjauh → `leaf1`
3. Dari `leaf1`, cari node terjauh → `leaf2`
4. `leaf1` ke `leaf2` adalah **diameter** pohon
5. Titik tengah diameter = **center** pohon

Bug yang Ditemukan

Bug 1 — `FindCenter` mengeset `maxHeight` dan `maxHeightNode` sebagai variabel lokal, tapi `farthestNode` membaca variabel **global** dengan nama yang sama. Akibatnya reset tidak berpengaruh ke global.

Bug 2 — Ketika `pathSize % 2 == 1` (path panjang ganjil), kode mencetak `path[pathSize/2] * -1` — dikalikan -1, ini jelas salah.

Bug 3 — `farthestNode` dipanggil pertama kali dari node `0`, tapi tree dibangun dengan node `1, 2, 3` dan tidak ada node `0` terhubung ke `1` (sebenarnya ada edge `1-0`, jadi `0` ada). Ini oke, tapi perlu diwaspadai kalau struktur tree berubah.

Kode Setelah Diperbaiki

Cell 1 — Inisialisasi global:

```
# To create tree
tree = {}
path = []
maxHeight, maxHeightNode = -1, -1
```

Cell 2 — getDiameterPath (tidak ada bug):

```
def getDiameterPath(vertex, targetVertex, parent, path):
    if (vertex == targetVertex):
        path.append(vertex)
        return True

    for i in range(len(tree[vertex])):
        if (tree[vertex][i] == parent):
            continue
        if (getDiameterPath(tree[vertex][i], targetVertex, vertex, path)):
            path.append(vertex)
```

```
            return True
    return False
```

Cell 3 — farthestNode (tidak ada bug):

```
def farthestNode(vertex, parent, height):
    global maxHeight, maxHeightNode

    if (height > maxHeight):
        maxHeight = height
        maxHeightNode = vertex

    if (vertex in tree):
        for i in range(len(tree[vertex])):
            if (tree[vertex][i] == parent):
                continue
            farthestNode(tree[vertex][i], vertex, height + 1)
```

Cell 4 — addedge (tidak ada bug):

```
def addedge(a, b):
    if (a not in tree):
        tree[a] = []
    tree[a].append(b)

    if (b not in tree):
        tree[b] = []
    tree[b].append(a)
```

Cell 5 — FindCenter, ini yang utama diperbaiki:

```
def FindCenter(n):
    global maxHeight, maxHeightNode

    # Bug 1 diperbaiki: pakai global, bukan variabel lokal
    maxHeight = -1
    maxHeightNode = -1

    farthestNode(0, -1, 0)
    leaf1 = maxHeightNode

    # Reset lagi untuk pencarian kedua
    maxHeight = -1
    farthestNode(leaf1, -1, 0)

    leaf2 = maxHeightNode

    path = []
    getDiameterPath(leaf1, leaf2, -1, path)

    pathSize = len(path)

    if (pathSize % 2 == 1):
        # Bug 2 diperbaiki: hapus * -1, cukup cetak node tengahnya
        print(path[int(pathSize / 2)])
    else:
        print(path[int(pathSize / 2)], ", ", path[int((pathSize - 1) / 2)], sep="", end="")
```

Cell 6 — Main (tidak ada bug):

```
N = 4

tree = {}
addedge(1, 0)
addedge(1, 2)
addedge(1, 3)

FindCenter(N)
```

Output

Struktur tree-nya seperti ini:

```
    0
    |
    1
```

```
  / \
 2   3
```

Node `1` terhubung ke `0`, `2`, dan `3`. Diameter pohon ini misalnya dari `0` ke `2` atau `0` ke `3`, panjang path = 3 node. Titik tengahnya adalah node `1`.

```
1
```

Node `1` adalah center dari pohon tersebut karena posisinya paling sentral — semua node lain berjarak maksimal 1 langkah darinya.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/